# Automated Corporate Visitor & Security Gate Manager

## Python Programming Project

**Student Name:** Mehana Mathew
**Course:** MBA **Subject:** Programming with Python
**Project Type:** Business Problem Solution using Python - CIA II
**Instructor:** Dr. Helen

---

## 1. Abstract

Modern corporate organizations receive numerous visitors each day, including clients, interview candidates, vendors, consultants, and service personnel. Managing these visitors using traditional manual logbooks can be inefficient, prone to human error, and difficult to maintain or retrieve when required. Such manual systems also make it challenging for security personnel to monitor visitor movements and maintain accurate records of entry and exit times.

The **Automated Corporate Visitor & Security Gate Manager** is a Python-based system designed to digitize and streamline the visitor management process within corporate environments. The system enables security staff to register visitors, generate unique visitor passes, record entry and exit timestamps automatically, and maintain structured visitor logs for future reference. The implementation of this system utilizes fundamental programming concepts such as control structures, loops, modular functions, object-oriented programming through classes, data structures including lists and dictionaries, exception handling for robust error management, and file handling for persistent data storage.

By automating visitor registration and tracking, the system enhances operational efficiency, improves record management, and strengthens corporate security monitoring. The project demonstrates how core Python programming techniques can be applied to solve real-world business problems through the development of a structured and modular software solution.

## 2. Problem Statement

Many corporate organizations continue to rely on manual visitor logbooks maintained at security gates to record the entry and exit of visitors. In this traditional system, security personnel manually record visitor details such as name, purpose of visit, employee to be met, and entry or exit time. While this method has been widely used, it suffers from several operational limitations.

Manual visitor registers are time-consuming to maintain and can lead to delays during peak visiting hours. Since records are written manually, they are also prone to human errors, incomplete information, and illegible entries. Additionally, retrieving past visitor records from physical logbooks can be difficult and inefficient, particularly when searching for specific visitors or analyzing historical data. There is also a risk of data loss or damage due to physical wear, misplacement of records, or lack of proper archiving.

Another major limitation of manual systems is the lack of real-time monitoring. Security personnel cannot easily track which visitors are currently inside the building, and entry or exit times may not always be recorded accurately.

To address these challenges, there is a need for a computerized visitor management system that can automate visitor registration, generate visitor passes, record entry and exit times accurately, maintain organized digital records, and allow quick retrieval of visitor information when required.

The proposed **Automated Corporate Visitor & Security Gate Manager** aims to solve these problems by implementing a Python-based system that digitizes the visitor management process and improves efficiency, security monitoring, and record management within corporate environments.

## 3. Objectives of the Project

The primary objective of this project is to design and implement an automated system for managing visitor entry and exit within a corporate environment. The system aims to replace traditional manual visitor registers with a digital solution that improves efficiency, accuracy, and security monitoring.

The specific objectives of the project are:

- To automate the visitor registration process at corporate security gates.
- To maintain structured records of employees who can be visited within the organization.

- To automatically generate unique visitor passes for identification and tracking purposes.
- To record visitor entry and exit timestamps accurately using system-generated time data.
- To store and manage visitor information in digital files for reliable record keeping.
- To provide functionality for retrieving and displaying visitor logs when required.
- To enable monitoring of visitor movement within the premises for improved security management.
- To demonstrate the use of core Python programming concepts such as object-oriented programming, functions, data structures, exception handling, and file management in solving a real-world business problem.

## 4. System Features

The **Automated Corporate Visitor & Security Gate Manager** provides several functional features designed to improve visitor tracking and security monitoring within a corporate environment. The key system features are described below:

1. **Visitor Registration** Allows security personnel to register new visitors by entering essential details such as visitor ID, name, phone number, purpose of visit, and the employee they intend to meet.
2. **Automatic Visitor Pass Generation** Generates a unique visitor pass number automatically for each visitor, enabling easy identification and tracking during their visit.
3. **Entry Time Recording** Automatically records the exact date and time when a visitor enters the premises using system-generated timestamps.
4. **Exit Time Recording** Updates the visitor record with the time of exit when the visitor leaves the building, ensuring accurate tracking of visitor duration.
5. **Employee Management** Allows the system to store and maintain employee information so that visitors can be linked to specific employees within the organization.
6. **Visitor Log Display** Provides functionality to display all stored visitor records, allowing security personnel to review visitor history when needed.
7. **Visitor Search Capability** Enables searching for specific visitor records using identifiers such as visitor ID or name, improving record retrieval efficiency.
8. **Exception Handling and Input Validation** Ensures that invalid inputs such as incorrect phone numbers or non-numeric menu selections are handled properly to prevent system crashes.
9. **File-Based Data Storage** Stores visitor and employee information in text files, allowing the system to maintain persistent records even after the program is closed.

## 5. ER Diagram

The Entity–Relationship (ER) Diagram represents the structure of the data used in the **Automated Corporate Visitor & Security Gate Manager**. It identifies the key entities in the system, their attributes, and the relationships between them. The ER model helps in understanding how visitor information, employee details, and security records interact within the system.

### Entities

The system consists of the following primary entities:

**1. Visitor** This entity stores information about individuals visiting the corporate premises.

Attributes:

- Visitor_ID (Primary Key)
- Visitor_Name
- Phone_Number
- Purpose_of_Visit

**2. Employee** This entity stores information about employees within the organization who may receive visitors.

Attributes:

- Employee_ID (Primary Key)
- Employee_Name
- Department

**3. Visit_Log** This entity records each visit made by a visitor, including entry and exit details.

Attributes:

- Log_ID (Primary Key)
- Visitor_ID (Foreign Key)
- Employee_ID (Foreign Key)
- Pass_Number
- Entry_Time
- Exit_Time

**4. Security_Gate** This entity represents the security checkpoint where visitor entry and exit are recorded.

Attributes:

- Gate_ID
- Security_Staff_Name
- Shift

### Relationships

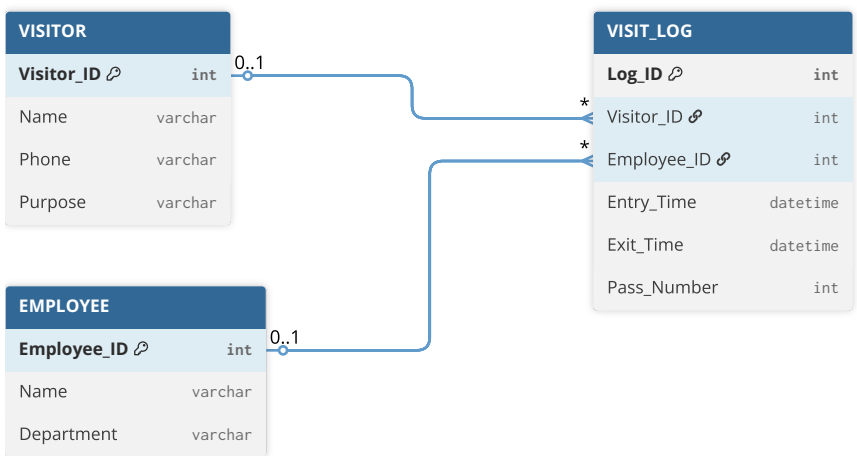
The entities in the system are connected through the following relationships:

- **Visitor → Visit_Log** A single visitor can have multiple visit records. Each visit generates a new log entry.
- **Employee → Visit_Log** An employee can receive multiple visitors. Each visit is recorded in the visit log and linked to the respective employee.
- **Security_Gate → Visit_Log** The security gate manages the visitor entry and exit process, and each visitor record is logged through the security system.

These relationships ensure that visitor activities are properly tracked and linked to both the employee being visited and the security checkpoint responsible for managing entry and exit.

```
from IPython.display import SVG, display

display(SVG("/content/Untitled.svg"))
```




## 6. Process Flow Diagram

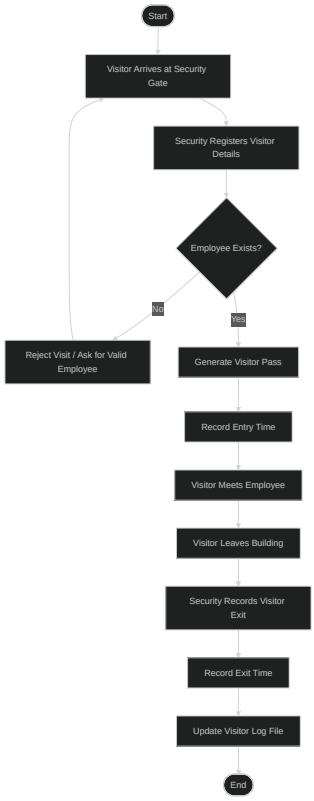
The Process Flow Diagram illustrates the sequence of operations performed by the **Automated Corporate Visitor & Security Gate Manager** when managing visitor entry and exit at the corporate security gate. The process begins when a visitor arrives at the premises and ends when the visitor exits and the visit record is updated in the system.

### Process Steps

1. **Visitor Arrival** The process begins when a visitor arrives at the corporate security gate and requests entry into the premises.
2. **Visitor Registration** The security personnel enters the visitor's details into the system, including visitor ID, name, phone number, purpose of visit, and the employee to be met.
3. **Employee Verification** The system checks whether the specified employee exists in the employee records. This ensures that visitors are only registered for valid employees.

4. **Visitor Pass Generation** Once the visitor details are verified, the system automatically generates a unique visitor pass number for identification and tracking.
5. **Entry Time Recording** The system records the exact entry timestamp using the system clock to maintain accurate visitor tracking.
6. **Visitor Meeting Process** After registration, the visitor proceeds to meet the designated employee within the organization.
7. **Visitor Exit Request** When the visitor leaves the premises, the security personnel records the visitor's exit using the visitor ID.
8. **Exit Time Recording** The system updates the visitor record with the exact exit timestamp.
9. **Visitor Log Update** Finally, the system updates the visitor log file to maintain a complete digital record of the visit.

This structured process ensures accurate visitor tracking, improves security monitoring, and maintains reliable records of all visitor activities within the corporate environment.




Process Flow Diagram

## 7. Python Implementation

```
import datetime
import random
```

### 7.1 Data Structures (Containers)

The system stores data using Python containers.

Two lists are used:

visitors → temporarily stores visitor objects
employees → stores employee objects

Lists allow us to store multiple records and easily access them using loops.

```
visitors = []
employees = []
```

### 7.2 Visitor Class

The Visitor class represents each visitor entering the building.

Attributes stored in the class:

Visitor ID
Visitor Name
Phone Number
Purpose of Visit
Employee to Meet
Pass Number
Entry Time
Exit Time

When a visitor is registered, an object of this class is created.

```
class Visitor:

    def __init__(self, vid, name, phone, purpose, employee):
        self.vid = vid
        self.name = name
        self.phone = phone
        self.purpose = purpose
        self.employee = employee

        self.pass_number = "PASS" + str(random.randint(1000,9999))

        self.entry_time = str(datetime.datetime.now())

        self.exit_time = "Not Exited"
```

## 7.3 Employee Class

The Employee class stores information about company employees.

Visitors must specify which employee they are meeting.

Attributes:

Employee ID
Employee Name
Department

```
class Employee:

    def __init__(self, eid, name, department):
        self.eid = eid
        self.name = name
        self.department = department
```

## 7.4 Employee Management Module

The Employee Management module allows the system to store employees that visitors can meet.

This ensures that visitors are linked to a specific employee in the company.

### Purpose

- Maintain a list of employees
- Allow visitors to specify which employee they are meeting
- Store employee information in a file

### Employee Information Stored

- Employee ID
- Employee Name
- Department

When a new employee is added:

1. An Employee object is created

2. It is stored in the employee list
3. The information is saved in the employees.txt file

```
def add_employee():

    try:
        eid = input("Enter Employee ID: ")
        name = input("Enter Employee Name: ")
        department = input("Enter Department: ")

        emp = Employee(eid, name, department)

        employees.append(emp)

        file = open("employees.txt", "a")
        file.write(f"{eid},{name},{department}\n")
        file.close()

        print("Employee added successfully")

    except Exception as e:
        print("Error adding employee:", e)
```

## 7.5 File Handling

The system saves visitor data into a text file.

File used: visitors.txt

Each visitor record is stored in a single line in comma-separated format.

This allows the company to maintain a permanent visitor log.

```
def save_visitor(visitor):

    file = open("visitors.txt","a")

    file.write(
        f"{visitor.vid},{visitor.name},{visitor.phone},{visitor.purpose},{visitor.employee},{visitor.pass_number},{vi
    )

    file.close()
```

## 7.6 Visitor Registration

This function registers a new visitor.

Steps performed:

1. Security enters visitor details
2. Visitor object is created
3. Visitor pass is generated
4. Entry time is recorded automatically
5. Visitor data is stored in a file

Exception handling is used to handle invalid phone numbers.

```
def register_visitor():

    try:
        vid = input("Visitor ID: ")
        name = input("Visitor Name: ")
        phone = int(input("Phone Number: "))
        purpose = input("Purpose of Visit: ")
        employee = input("Employee to Meet: ")

        visitor = Visitor(vid,name,phone,purpose,employee)

        visitors.append(visitor)

        save_visitor(visitor)
```

```
            print("Visitor Registered Successfully")
            print("Visitor Pass:", visitor.pass_number)

        except ValueError:
            print("Invalid phone number entered")
```

## 7.7 Visitor Exit System

This function records when a visitor leaves the building.

Steps:

1. Security enters visitor ID
2. System searches the visitor record
3. Exit time is recorded
4. Visitor log is updated

```
def visitor_exit():

    vid = input("Enter Visitor ID: ")

    try:
        file = open("visitors.txt","r")
        lines = file.readlines()
        file.close()

        file = open("visitors.txt","w")

        found = False

        for line in lines:

            data = line.strip().split(",")

            if data[0] == vid:
                data[7] = str(datetime.datetime.now())
                line = ",".join(data) + "\n"
                found = True

            file.write(line)

        file.close()

        if found:
            print("Exit time recorded")

        else:
            print("Visitor not found")

    except FileNotFoundError:
        print("Visitor file not found")
```

## 7.8 Display Visitor Logs

This function displays all visitor records stored in the system.

It reads the visitors.txt file and prints all records.

```
def display_visitors():

    try:
        file = open("visitors.txt","r")

        print("\nVisitor Log")

        for line in file:
            print(line.strip())

        file.close()

    except:
        print("No visitor records available")
```

## 7.9 Main Menu System

The menu system allows the security guard to select different operations.

Options include:

1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit System

A while loop keeps the system running continuously.

```
def menu():

    while True:

        print("\nSecurity Gate Manager")
        print("1 Register Visitor")
        print("2 Visitor Exit")
        print("3 Display Visitors")
        print("4 Add Employee")
        print("5 Exit")

        try:

            choice = int(input("Enter choice: "))

            if choice == 1:
                register_visitor()

            elif choice == 2:
                visitor_exit()

            elif choice == 3:
                display_visitors()

            elif choice == 4:
                add_employee()

            elif choice == 5:
                print("System Closed")
                break

            else:
                print("Invalid Choice")

        except ValueError:
            print("Please enter a number")
```

## 7.10 Running the System

```
def menu():

    while True:

        print("\nSecurity Gate Manager")
        print("1 Register Visitor")
        print("2 Visitor Exit")
        print("3 Display Visitors")
        print("4 Add Employee")
        print("5 Exit")

        try:

            choice = int(input("Enter choice: "))

            if choice == 1:
                register_visitor()

            elif choice == 2:
```

```
                    visitor_exit()

                elif choice == 3:
                    display_visitors()

                elif choice == 4:
                    add_employee()

                elif choice == 5:
                    print("System Closed")
                    break

                else:
                    print("Invalid Choice")

            except ValueError:
                print("Please enter a number")
```

## 8. Example System Execution Using the Menu

The **Automated Corporate Visitor & Security Gate Manager** is implemented as a menu-driven application. Security personnel interact with the system by selecting options from the main menu. Each menu option triggers a specific module of the system, such as visitor registration, visitor exit recording, employee management, or viewing visitor logs.

The following example demonstrates a complete workflow using the menu system.

### Step 1: Running the System

When the program starts, the main menu is displayed.

```
Security Gate Manager

1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit
```

The user selects an option by entering the corresponding number.

---

### Step 2: Adding an Employee

Before registering visitors, the security personnel can add an employee to the system.

```
Enter choice: 4

Enter Employee ID: E101
Enter Employee Name: Ajay
Enter Department: Human Resources

Employee added successfully
```

This employee record is stored in the **employees.txt** file.

---

### Step 3: Registering a Visitor

Next, the security personnel registers a visitor.

```
Enter choice: 1

Visitor ID: V101
Visitor Name: Rahul
Phone Number: 9876543210
Purpose of Visit: Interview
Employee to Meet: Ajay
```

The system generates a visitor pass and records the entry time.

```
Visitor Registered Successfully
Visitor Pass: PASS4821
Entry Time: 2026-03-11 10:40:12
```

The following record is automatically written to **visitors.txt**:

```
V101,Rahul,9876543210,Interview,Ajay,PASS4821,2026-03-11 10:40:12,Not Exited
```

---

### Step 4: Visitor Exit

When the visitor leaves the premises, the security personnel records the exit time.

```
Enter choice: 2

Enter Visitor ID: V101
Exit time recorded successfully
```

The system updates the visitor record in **visitors.txt**:

```
V101,Rahul,9876543210,Interview,Ajay,PASS4821,2026-03-11 10:40:12,2026-03-11 11:15:08
```

---

### Step 5: Displaying Visitor Logs

The security personnel can view all visitor records stored in the system.

```
Enter choice: 3
```

Example output:

```
Visitor Log

V101,Rahul,9876543210,Interview,Ajay,PASS4821,2026-03-11 10:40:12,2026-03-11 11:15:08
V102,Neha,9876541234,Meeting,Rahul,PASS3948,2026-03-11 11:30:55,Not Exited
```

This feature allows security staff to quickly review visitor history.

---

### Summary of Menu-Based Workflow

The complete workflow using the menu system follows this sequence:

1. Start the system and display the main menu
2. Add employees who may receive visitors
3. Register visitors and generate visitor passes
4. Record visitor exits when visitors leave
5. Display visitor logs for monitoring and record keeping

This menu-driven structure ensures that the system remains simple, organized, and easy for security personnel to operate.

## Adding an Employee

```
menu()
```

```
Security Gate Manager
1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit
Enter choice: 4
Enter Employee ID: E101
Enter Employee Name: Mehana
Enter Department: Human Resources
Employee added successfully
```

```
Security Gate Manager
1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit
Enter choice: 1
Visitor ID: V101
Visitor Name: Anant
Phone Number: 9991119991
Purpose of Visit: Interview
Employee to Meet: Mehana
Visitor Registered Successfully
Visitor Pass: PASS6855

Security Gate Manager
1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit
Enter choice: 5
System Closed
```

## Adding a visiter

```
menu()
```

```
Security Gate Manager
1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit
Enter choice: 3

Visitor Log
V101,Anant,9991119991,Interview,Mehana,PASS6855,2026-03-11 07:22:53.238947,Not Exited

Security Gate Manager
1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit
Enter choice: 5
System Closed
```

## Exiting the Visiter

```
menu()
```

```
Security Gate Manager
1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit
Enter choice: 2
Enter Visitor ID: V101
Exit time recorded

Security Gate Manager
1 Register Visitor
2 Visitor Exit
3 Display Visitors
4 Add Employee
5 Exit
Enter choice: 3

Visitor Log
V101,Anant,9991119991,Interview,Mehana,PASS6855,2026-03-11 07:22:53.238947,2026-03-11 07:23:23.520802

Security Gate Manager
1 Register Visitor
2 Visitor Exit
3 Display Visitors
```

```
    4 Add Employee
    5 Exit
    Enter choice: 5
    System Closed
```

## Visitor Data File (visitors.txt)

The **visitors.txt** file is used to store all visitor records generated by the system. It acts as a simple persistent storage mechanism so that visitor data remains available even after the program is closed.

Each visitor entry is stored as a **single line in comma-separated format (CSV-like structure)**. This allows the system to easily read, update, and display visitor information using Python file handling operations.

### Structure of a Visitor Record

Each record in the file contains the following fields:

| Field | Description |
|---|---|
| Visitor_ID | Unique identifier for the visitor |
| Visitor_Name | Name of the visitor |
| Phone_Number | Contact number of the visitor |
| Purpose_of_Visit | Reason for visiting the organization |
| Employee_to_Meet | Employee being visited |
| Pass_Number | Automatically generated visitor pass |
| Entry_Time | Timestamp when visitor entered |
| Exit_Time | Timestamp when visitor exited |

### Example Contents of `visitors.txt`

```
V101,Rahul,9876543210,Interview,Ajay,PASS4821,2026-03-11 10:40:12,2026-03-11 11:15:08
V102,Neha,9876541234,Meeting,Rahul,PASS3948,2026-03-11 11:30:55,Not Exited
V103,Amit,9876509876,Vendor Delivery,Karan,PASS2751,2026-03-11 12:05:14,2026-03-11 12:45:33
```

In this example:

- **Visitor V101** completed the visit and has both entry and exit timestamps.
- **Visitor V102** is still inside the building because the exit time is marked as **Not Exited**.
- **Visitor V103** completed a delivery visit and exited later.

This structure makes it easy to:

- track visitor history
- identify visitors currently inside the building
- analyze visitor activity if needed.

## Employee Data File (employees.txt)

The **employees.txt** file is used to store information about employees who work within the organization and may receive visitors. This file acts as a simple persistent storage system that allows the application to maintain employee records even after the program is closed.

Whenever a new employee is added using the **Add Employee** option from the system menu, the employee's details are written to the `employees.txt` file using Python file handling operations. These records are later used when registering visitors to ensure that each visitor is associated with a valid employee in the organization.

### Structure of an Employee Record

Each employee record is stored as a **single line in comma-separated format**, making it easy for the program to read and process the data.

Each line in the file contains the following fields:

| Field | Description |
|---|---|
| Employee_ID | Unique identifier assigned to the employee |
| Employee_Name | Name of the employee |
| Department | Department in which the employee works |

### Example Contents of `employees.txt`

```
E101,Ajay,Human Resources
E102,Rahul,Finance
E103,Neha,Information Technology
```

### How the File is Used in the System

The `employees.txt` file supports several important operations within the system:

- It stores employee records added through the **Employee Management module**.
- It allows the system to verify that a visitor is meeting a valid employee.
- It helps maintain a structured relationship between **visitors and employees**.
- It ensures employee data is preserved even after the application is closed.

By storing employee data in a file, the system provides a simple yet effective method for maintaining organizational records without requiring a full database system.

## 9. Technologies Used

The development of the **Automated Corporate Visitor & Security Gate Manager** involves several technologies and programming concepts that enable efficient implementation of the visitor management system. The key technologies used in this project are described below.

### Programming Language

**Python**

Python is used as the primary programming language for developing the system. It provides a simple syntax, strong support for object-oriented programming, and built-in libraries that make it suitable for rapid application development.

### Development Platform

**Jupyter Notebook**

The project is implemented and documented using Jupyter Notebook. This platform allows the integration of explanatory text (Markdown) and executable Python code in a single environment, making it ideal for demonstrating both the implementation and explanation of the system.

### Data Storage

**Text Files**

Visitor and employee records are stored using text files. File handling operations are used to write, read, and update records in these files, ensuring that visitor information is stored persistently even after the program terminates.

### Python Libraries Used

- **datetime** – Used to capture and record the exact entry and exit timestamps of visitors automatically.
- **random** – Used to generate unique visitor pass numbers for identification and tracking purposes.

### Programming Concepts Applied

The project demonstrates several core programming concepts, including:

- **Classes and Objects** – Used to represent real-world entities such as visitors and employees using object-oriented programming.
- **Functions (Modularity)** – The program is divided into reusable functions such as visitor registration, exit recording, and employee management.
- **Data Structures (Lists)** – Lists are used to store collections of visitor and employee objects during program execution.
- **Loops** – Loops are used to maintain continuous system operation and process multiple records.
- **Control Structures** – Conditional statements are used to implement decision-making within the system.
- **Exception Handling** – Error handling mechanisms are used to manage invalid inputs and prevent system crashes.
- **File Handling** – Used to store, retrieve, and update visitor and employee data in external files.

## 10. Results

The implementation of the **Automated Corporate Visitor & Security Gate Manager** successfully demonstrates how a digital system can replace traditional manual visitor registers in corporate environments. The developed application was tested using various visitor and

employee records, and the system performed all intended operations effectively.

The system successfully supports visitor registration by allowing security personnel to enter visitor details such as visitor ID, name, phone number, purpose of visit, and the employee being visited. Upon registration, the system automatically generates a unique visitor pass number and records the visitor's entry time using system-generated timestamps.

The system also allows security personnel to record visitor exits by entering the visitor ID, after which the system updates the corresponding record with the exit time. All visitor information is stored in external text files, ensuring that the data is preserved even after the application is closed.

Additionally, the system provides functionality to display stored visitor logs, allowing security staff to review visitor records when required. Exception handling mechanisms ensure that invalid inputs are managed properly, preventing system crashes and improving reliability.

Overall, the system improves operational efficiency by reducing the time required for visitor registration, minimizes human errors associated with manual record keeping, and enhances the ability of security personnel to monitor visitor activities within the organization.

## 11. Conclusion

The **Automated Corporate Visitor & Security Gate Manager** successfully demonstrates how a digital solution can be developed to replace traditional manual visitor registration systems used in corporate environments. By automating the process of visitor registration, entry and exit time recording, and visitor log maintenance, the system improves the efficiency and reliability of visitor management at corporate security gates.

The project highlights the practical application of Python programming concepts such as object-oriented programming, modular functions, control structures, data structures, exception handling, and file handling. Through the integration of these concepts, the system is able to manage visitor data efficiently while maintaining structured records that can be easily accessed and updated.

The developed system reduces the dependency on manual record keeping, minimizes the risk of human errors, and provides better tracking of visitor activities within the organization. It also demonstrates how simple programming tools can be used to address real-world business problems related to security monitoring and record management.

In the future, the system can be further enhanced by incorporating a graphical user interface (GUI) to improve usability, integrating a database management system for more efficient data storage and retrieval, implementing QR code or digital visitor passes for faster identification, and enabling real-time monitoring of visitors currently present within the premises. These improvements would further strengthen the effectiveness and scalability of the visitor management system.